

Hugging Face와 GPT: 생성형 AI 실전 가이드

트랜스포머 라이브러리 활용부터 나만의 챗봇 구현까지 (GenWorks 5주차)



“이론을 넘어, 내 컴퓨터에서 직접 돌아가는 GPT를 만듭니다.”

1. 환경 준비
(Setup)

2. 모델 로드
(Summoning)

3. 파라미터 제어
(Controlling)

4. 어플리케이션 구축
(Building)



학습 목표 (The Blueprint)

- ✓ transformers: 10만 개 이상의 모델 지원
- ✓ accelerate & torch: 하드웨어 가속 및 연산
- ✓ datasets: 데이터 관리 및 로드

환경 설정 (Setup)

```
$ ~  
$ pip install transformers datasets torch accelerate  
> pip install sentencepiece huggingface_hub
```

GPU 생명 신호 확인

```
import torch  
print(torch.cuda.is_available())  
print(torch.cuda.get_device_name(0))
```



VRAM 용량은 모델 선택의 절대적 기준입니다.

Hugging Face 생태계와 핵심 컴포넌트

오픈소스 NLP의 허브. 복잡한 신경망 코드를 3가지 핵심 클래스로 추상화합니다.

The Transformer Trinity



AutoTokenizer
텍스트 → 숫자 (Input)



AutoModelForCausalLM
추론 및 연산 (The Engine)



Pipeline
전처리 + 추론 + 후처리 (Interface)

Hugging Face Hub (Models / Datasets)

모델 로드 전략: 엔진 선택하기

	모델명	파라미터 크기	필요 VRAM	특징
1	GPT-2	124M	1~4GB	가볍고 빠름, 실습용
2	GPT-Neo	125M ~ 2.7B	1~6GB	GPT-3 아키텍처 복제
3	Qwen2.5	7B ~ 72B	14GB+	고성능, 다국어 지원

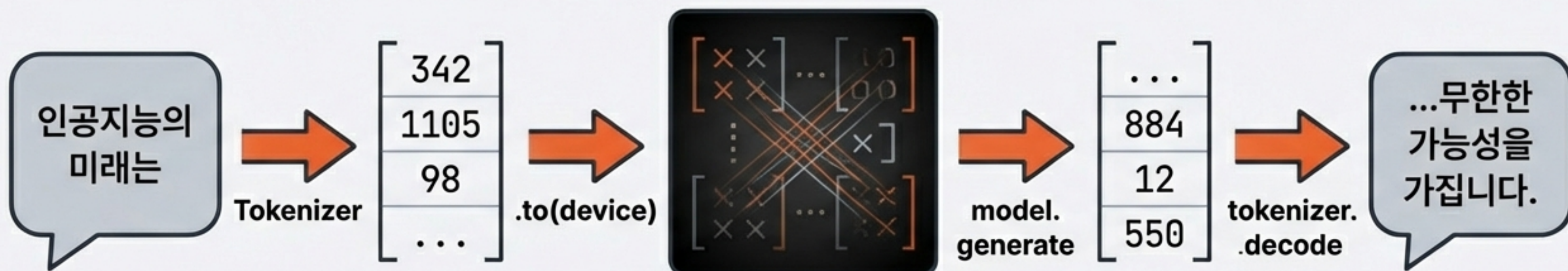
```
model = AutoModelForCausalLM.from_pretrained(  
    MODEL_NAME,  
    device_map='auto', # GPU 자동 할당  
    torch_dtype=torch.float16 # 메모리 절약 (FP16)  
)
```

핵심: 메모리 최적화

속도 2배 / 메모리 1/2

첫 번째 텍스트 생성 (Hello, AI)

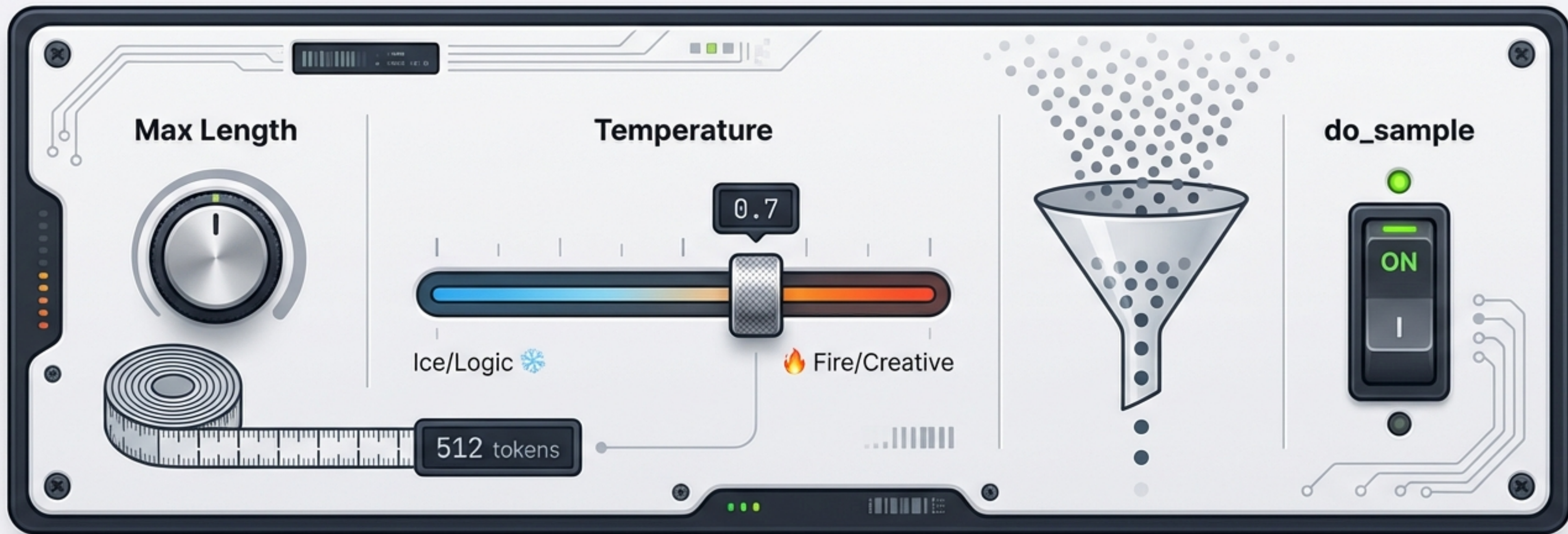
텍스트가 숫자로, 다시 텍스트로 변환되는 과정



```
inputs = tokenizer(text, return_tensors='pt').to(device)
outputs = model.generate(**inputs)
print(tokenizer.decode(outputs[0]))
```


생성 파라미터 해부: 제어 패널 (The Control Knobs)

`model.generate()` 내부의 파라미터는 AI의 성격을 결정합니다.



생성 길이 제한

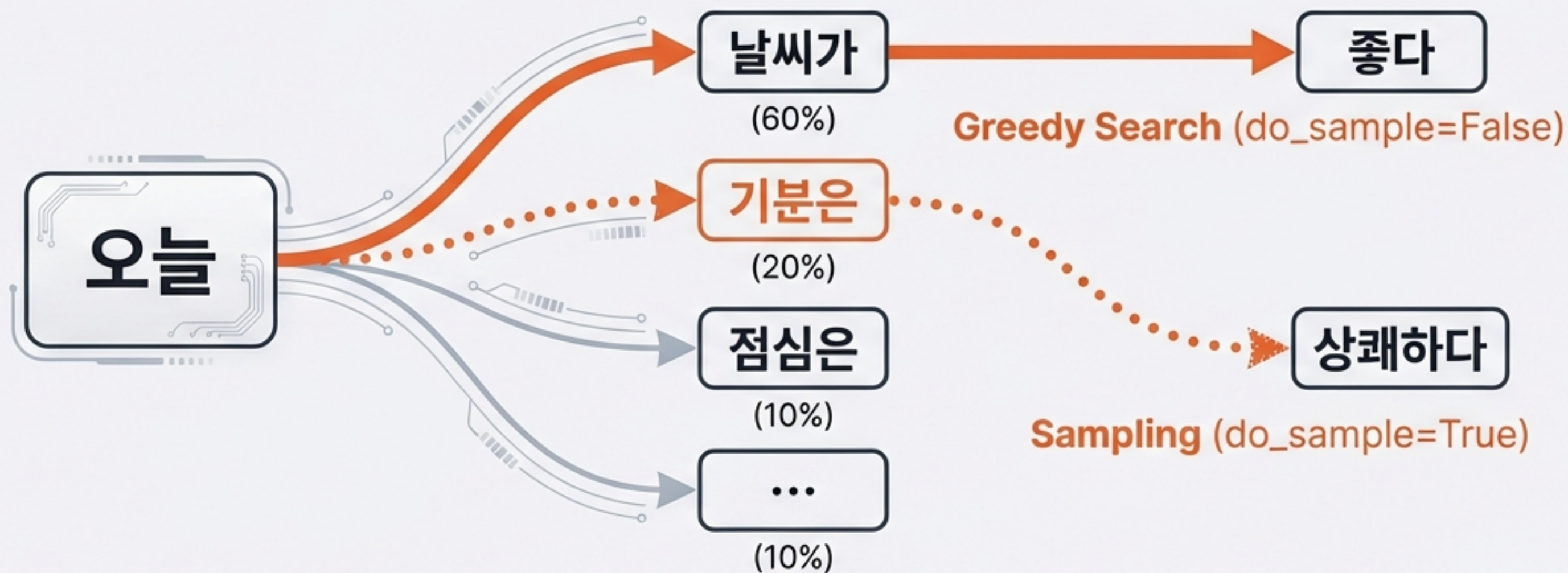
창의성 (0.1 ~ 1.0)

확률 필터링

True: 다양성 /
False: 고정

샘플링 전략: Greedy vs Sampling

다음을 예측하는 두 가지 방법



Greedy: 항상 확률이 가장 높은 단어 선택 (반복적, 안전함).

Sampling: 확률에 따라 다양한 단어 선택 (창의적, 인간적).

실습: Temperature(온도) 실험실

Prompt: "Artificial Intelligence is..."

Temperature 0.1 (Solid)



AI is a computer system. AI is a computer system. AI is a computer system...

지루함 / 반복 / 논리적

Temperature 0.7 (Liquid)



AI is transforming the world with new world with new technologies like deep learning.

균형 / 자연스러움 (권장)

Temperature 1.2 (Gas)

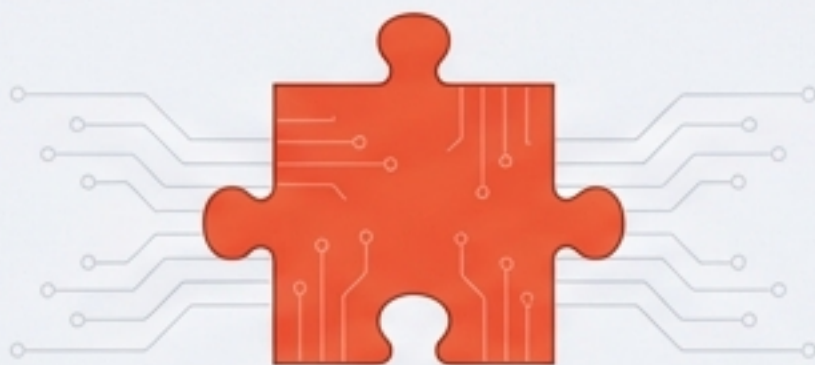


AI dreams electric sheep clouds algorithm fly into sky blue pizza.

혼란 / 환각 / 과도한 창의성

실전 응용 1: 문장 완성을 넘어 Q&A로

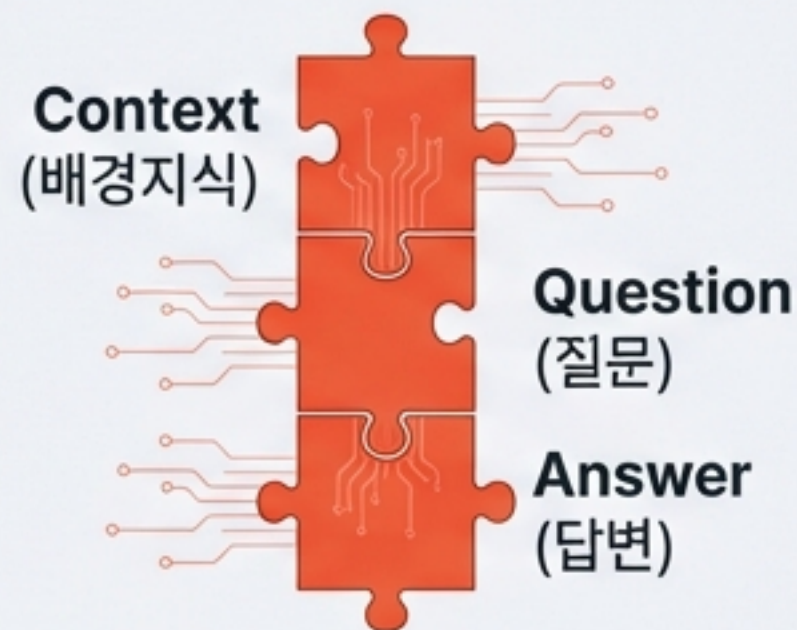
Text Completion



```
prompt = '대한민국의 수도는'  
generate(prompt)
```

→ 서울입니다.

Contextual Q&A (RAG Basis)



```
prompt = f'''
```

```
컨텍스트: 화성은 붉은 행성입니다.
```

```
질문: 화성의 색깔은?
```

```
답변:
```

```
'''
```

→ 붉은색입니다.

컨텍스트를 주입하여 환각(Hallucination)을 줄입니다.

실전 응용 2: 코딩(논리) vs 스토리텔링(창의)

목적에 따른 파라미터 튜닝

>_ Code Generation

Temp: 0.3 | Top-p: 0.9

```
def filter_even(numbers):  
    return [n for n in numbers  
            if n % 2 == 0]
```

정확성 중시 (Low Temp)



Story Generation

Temp: 0.9 | Repetition Penalty: 1.2

옛날 아주 먼 옛날, 마법사 알렉스는
용의 등 뒤에 숨겨진 비밀을 찾아...

창의성 중시 (High Temp)

성능 평가 및 모니터링

Generation Speed



Tokens per Second (t/s)

```
measure_generation_speed()
```

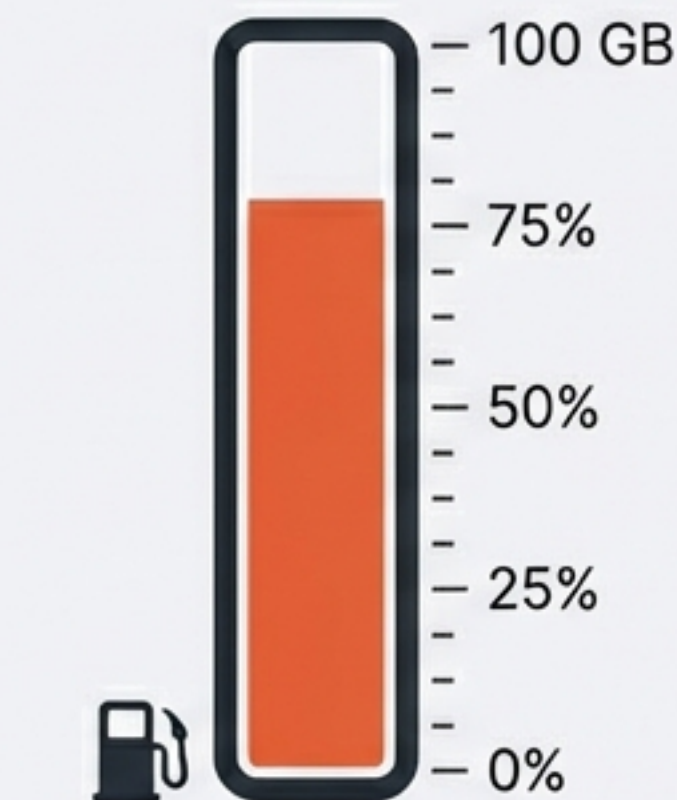
Pit Stop

Optimization Tip:

```
with torch.no_grad():  
    torch.cuda.empty_cache()
```

이 두 가지 명령어로 메모리 누수를 방지하세요.

Memory Load

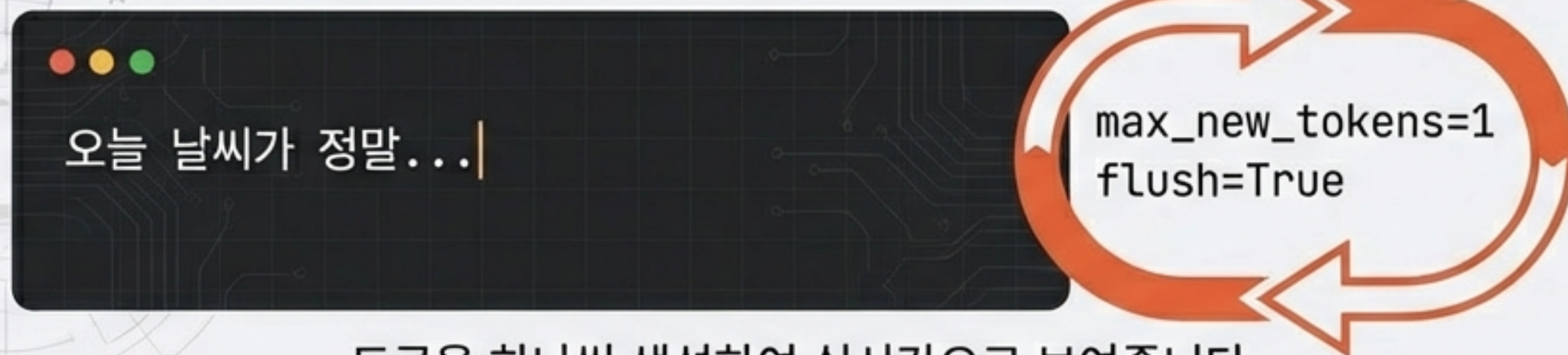


VRAM Usage (GB)

```
torch.cuda.memory_allocated()
```


고급 기능: 스트리밍(Streaming)과 배치(Batching)

Streaming (ChatGPT Style)



토큰을 하나씩 생성하여 실시간으로 보여줍니다.

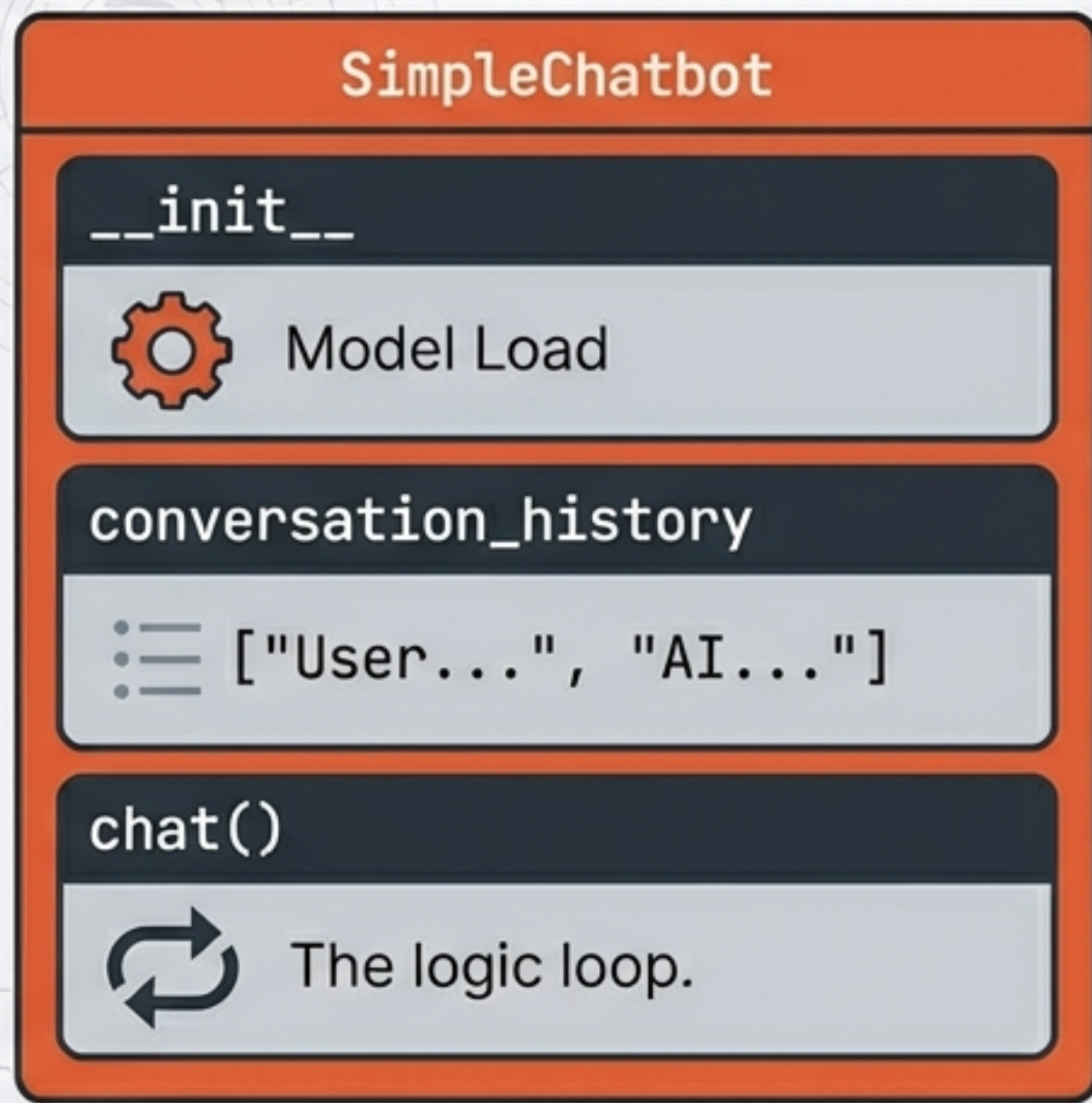
Batch Processing (Efficiency)



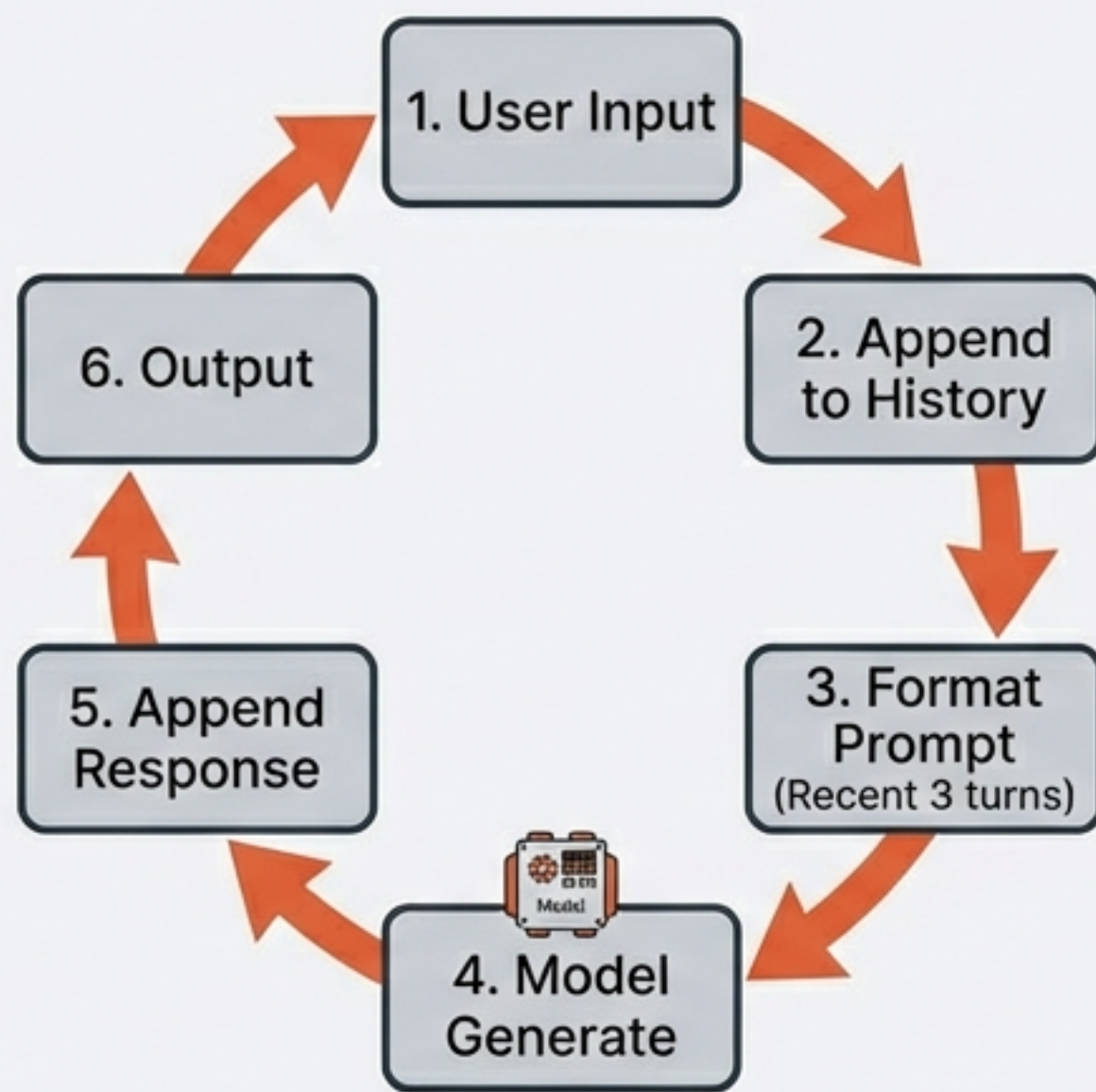
`padding=True`를 사용하여 여러 문장을 한 번에 처리합니다.

캡스톤 프로젝트: 나만의 챗봇(Chatbot) 만들기

CLASS ARCHITECTURE



THE LOGIC LOOP



문제 해결(Troubleshooting) 및 최적화



Symptom:
CUDA Out Of
Memory (OOM)

Prescription

- Batch Size 감소
- max_length 축소
- torch.float16 사용



Symptom:
이상한 문자 반복 /
품질 저하

Prescription

- Repetition Penalty
(1.1 ~ 1.2) 적용
- Temperature 낮추기

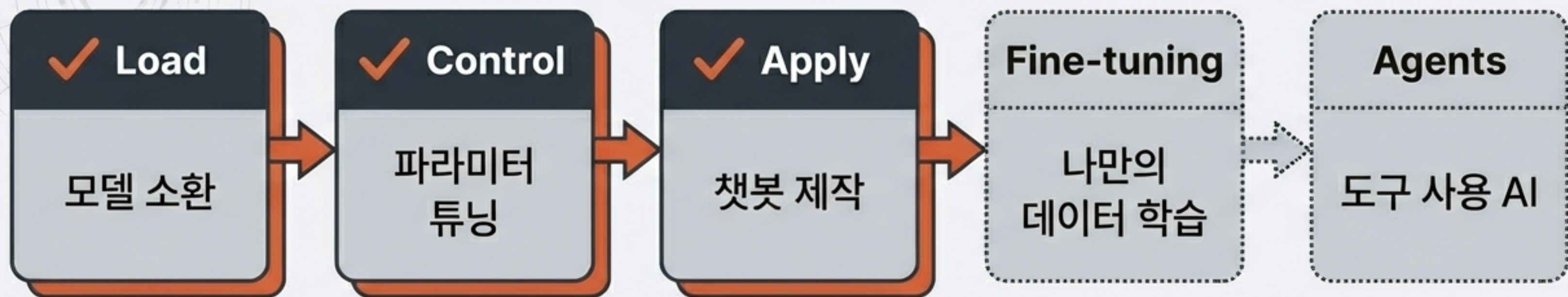


Maintenance:
메모리 관리

Prescription

- model.eval()
- torch.cuda.empty_cache()

요약 및 다음 단계 (Roadmap)



이제 여러분은 Hugging Face GPT를 자유자재로
다룰 수 있는 기본기를 갖추었습니다.